ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

ECYS | DITT
DESARROLLO DE TRANSFERENCIA TECNOLÓGICA

Día, Fecha:	Martes, 10/02/2026
Hora de inicio:	17:20

INTRODUCCIÓN A LA PROGRAMACIÓN Y COMPUTACIÓN 2

TOBÍAS RAFAEL ZAMORA SANTOS

Introducción a XML

Introducción a la Programación y Computación 2
Escuela de Ingeniería de Ciencias y Sistemas
Facultad de Ingeniería
Universidad de San Carlos de Guatemala

Agenda

Recordatorios

Lectura Enunciado del Proyecto

¿Qué es XML?

Partes y Características

Ventajas y Desventajas

Aplicaciones

Procesamiento de Datos XML

Recursos

Competencias a Desarrollar

Analizar actividades en archivos XML utilizando técnicas de lectura estructurada para identificar y extraer información relevante.

¿Qué es XML?

Es un metalenguaje que fue diseñado para estructurar, almacenar e intercambiar datos entre aplicaciones. Es un estándar ya que es extensible y puede ser utilizado independientemente de la plataforma.

Es el acrónimo de Extensible Markup Language. Es un lenguaje de marcado que define un conjunto de reglas para la codificación de documentos.

Es un metalenguaje que permite definir lenguajes de marcas desarrollado por el World Wide Web Consortium (W3C) utilizado para almacenar datos de forma legible.

¿Para qué nos sirve XML?

Sirve para brindar una estructura a la información que se encuentre en la web, para que la información pueda ser:

- Almacenada
- Procesada
- Visualizada
- Manipulada

Por aplicaciones, páginas o lo que se requiera.

Partes de XML

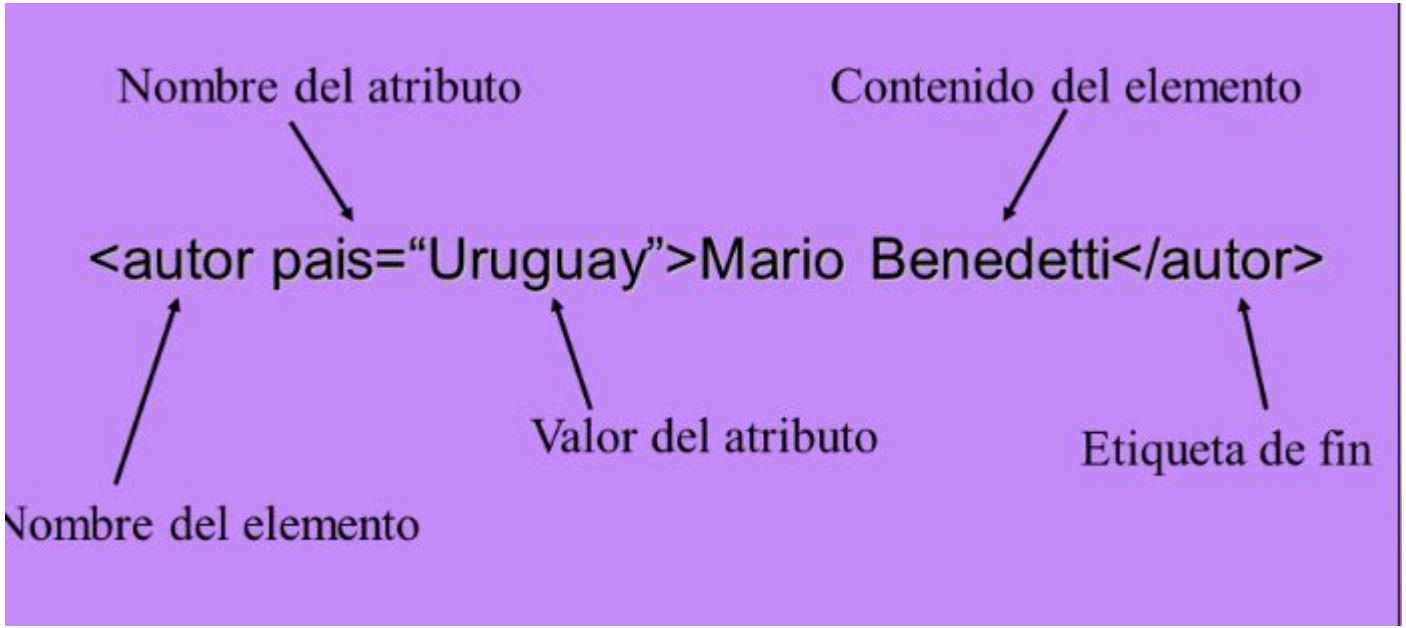
```xml
<?xml version="1.0" encoding="UTF-8"?>
<biblioteca>
  <libro>
    <titulo>La vida está en otra parte</titulo>
    <autor>Milan Kundera</autor>
    <fechaPublicacion año="1973"/>
  </libro>
  <libro>
    <titulo>Pantaleón y las visitadoras</titulo>
    <autor fechaNacimiento="28/03/1936">Mario Vargas Llosa</autor>
    <fechaPublicacion año="1973"/>
  </libro>
  <libro>
    <titulo>Conversación en la catedral</titulo>
    <autor fechaNacimiento="28/03/1936">Mario Vargas Llosa</autor>
    <fechaPublicacion año="1969"/>
  </libro>
</biblioteca>
```

Prólogo: En este se especifica que el documento en cuestión es XML, se define la versión y la codificación de caracteres que se utilizó en el archivo

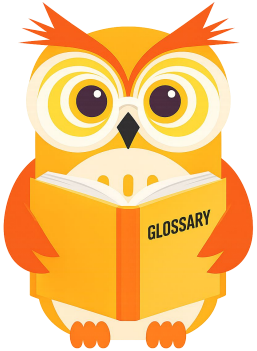
Elementos: Se tiene una etiqueta o tag inicial <> y una final </>.

Epílogo: Datos adicionales, como encriptación. Generalmente los tokens son agregados en este apartado. Este elemento es opcional.

Estructura de una Etiqueta



Características de XML



- **Extensible:** Cada persona puede crear las etiquetas que desee según el tipo de documento que se quiere crear. En otras palabras cada usuario podrá crear su propio lenguaje.

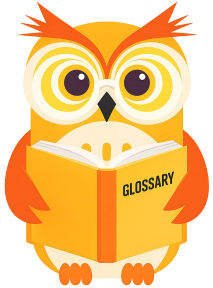
- **Estándar abierto:** XML es SGML, lo que significa que no es necesario saber programar, además de que existen muchas herramientas para su manipulación.

Características de XML

- **Eficiente:** La información debe ser transmitida una única vez para su manipulación.

- **Libre:** Nadie tiene una patente sobre él, ya que ha sido definido como un estándar internacional.

- **Manejable:** Tiene métodos para declarar y reforzar las estructuras que son utilizadas actualmente.



Ventajas de XML

- **Simplifica el intercambio de datos:** Dada la gran cantidad de aplicaciones que se tienen en la web, los desarrolladores pueden utilizar el mismo estándar para comunicar aplicaciones que a simple vista serían incompatibles.

- Los archivos XML se almacenan en forma de texto, lo que facilita la expansión y actualización de los mismos.

- **Aumenta la disponibilidad de datos:** El acceso a los archivos XML pueden darse de diferentes lugares, no únicamente desde HTML.

- Con XML se tiene la opción que los datos estén disponibles para todos los tipos de máquinas de lectura.

Desventajas de XML

- **Verboso:** XML tiende a ser más verboso que JSON, lo que puede hacer que los documentos sean más grandes y difíciles de leer.

- **Procesamiento más lento:** Debido a su estructura más compleja, el procesamiento de datos XML puede ser más lento en comparación con JSON.

- **Tamaño de archivo mayor:** En comparación con JSON, los documentos XML tienden a ser más grandes debido a su sintaxis y a la necesidad de incluir etiquetas de apertura y cierre para cada elemento. Esto puede llevar a un aumento en el consumo de ancho de banda y tiempo de transferencia, lo que podría ser una consideración importante en aplicaciones donde la velocidad de transferencia es crucial, como en servicios web o aplicaciones móviles.

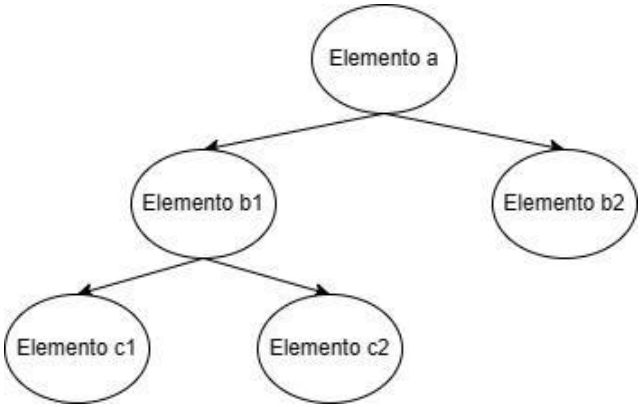
Aplicaciones XML

- Estructura HTML
- PDF está guardado en XML
- Microsoft Office guarda los datos como XML
- SOAP(Simple Object Access Protocol)
- Web Services de instituciones financieras
- Correos electrónicos

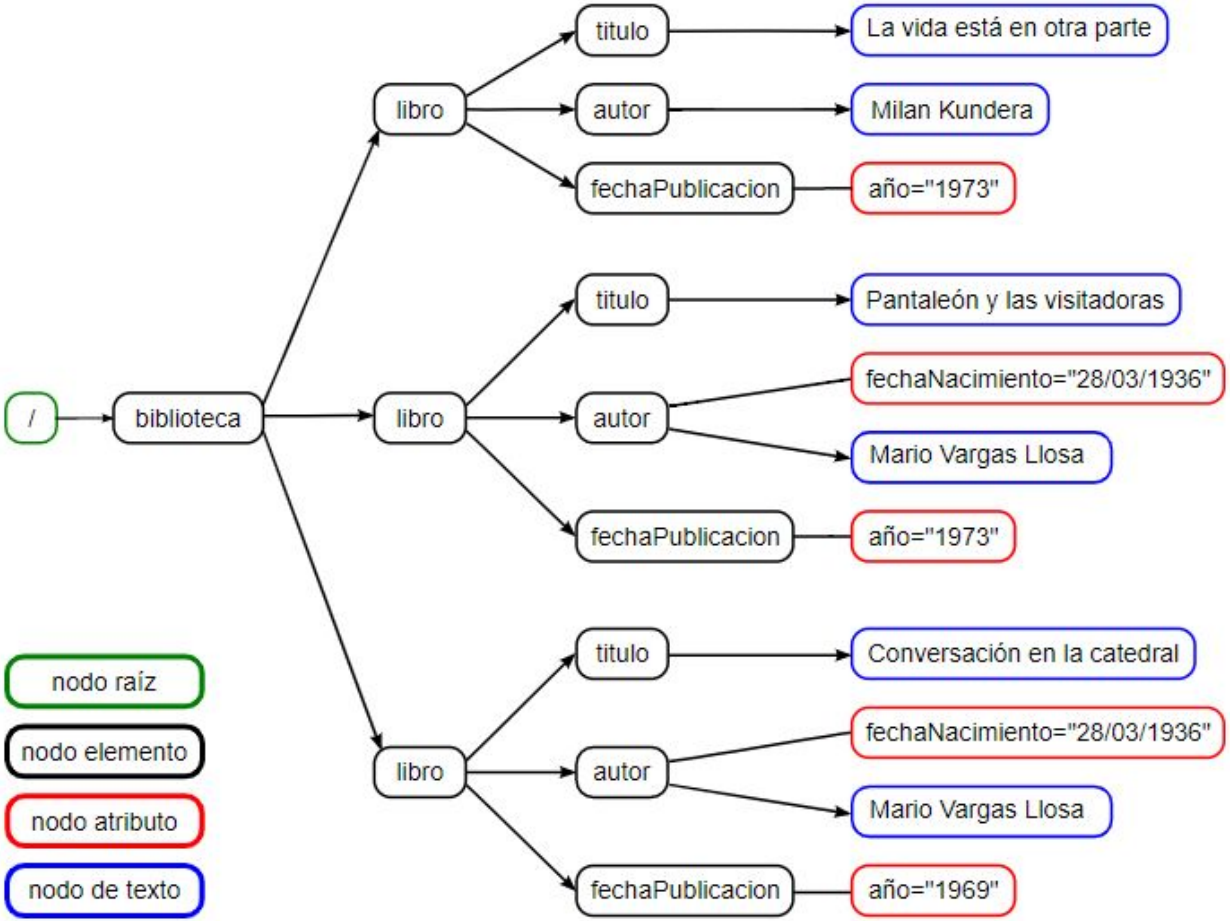
Árbol

Un árbol es una estructura jerárquica o secuencial formada por nodos, los cuales pueden ser de dos tipos: los nodos hijos y los nodos padre.

- Un padre puede tener muchos hijos
- Un hijo puede tener sus propios hijos, además de tener hermanos



```xml
<?xml version="1.0" encoding="UTF-8"?>
<biblioteca>
  <libro>
    <titulo>La vida está en otra parte</titulo>
    <autor>Milan Kundera</autor>
    <fechaPublicacion año="1973"/>
  </libro>
  <libro>
    <titulo>Pantaleón y las visitadoras</titulo>
    <autor fechaNacimiento="28/03/1936">Mario Vargas Llosa</autor>
    <fechaPublicacion año="1973"/>
  </libro>
  <libro>
    <titulo>Conversación en la catedral</titulo>
    <autor fechaNacimiento="28/03/1936">Mario Vargas Llosa</autor>
    <fechaPublicacion año="1969"/>
  </libro>
</biblioteca>
```


/
biblioteca
libro
titulo
La vida está en otra parte
autor
Milan Kundera
fechaPublicacion
año="1973"
libro
titulo
Pantaleón y las visitadoras
autor
fechaNacimiento="28/03/1936"
Mario Vargas Llosa
fechaPublicacion
año="1973"
libro
titulo
Conversación en la catedral
autor
fechaNacimiento="28/03/1936"
Mario Vargas Llosa
fechaPublicacion
año="1969"
nodo raíz
nodo elemento
nodo atributo
nodo de texto

Modelo DOM para procesar XML

El Modelo de Objetos del Documento, o DOM por sus siglas en inglés, es una librería del World Wide Web (W3C) para acceder y modificar documentos XML. Una implementación del DOM presenta los documento XML como un árbol, y permite al código cliente construir dichas estructuras desde cero para Crear nuestros propios árboles XML.

```
from xml.dom.minidom import parse, parseString

dom1 = parse('c:\\temp\\mydata.xml')  # parse an XML file by name

datasource = open('c:\\temp\\mydata.xml')
dom2 = parse(datasource)  # parse an open file

dom3 = parseString('<myxml>Some data<empty/> some more data</myxml>')
```

En el DOM, los documentos tienen una estructura lógica que es muy parecida a un árbol, sin embargo, no especifica que debe ser implementado como un árbol obligatoriamente.

El DOM es un modelo lógico que puede implementarse de cualquier manera que sea conveniente.

Una propiedad importante de los modelos de estructura del DOM es algo llamado “isomorfismo estructural”, el cual significa que si dos implementaciones cualesquiera del Modelo de Objetos del Documento se usan para crear una representación del mismo documento, ambas crearán el mismo modelo de estructura, con exactamente los mismos objetos y relaciones.

Origen del Modelo DOM

El DOM se originó como una especificación para permitir que los programas Java y los scripts de JavaScript fueran portables entre navegadores web.

La primera especificación del modelo surgió en 1997 llamada DOM level 1 del cual se realizó una actualización en 2000.

Se han realizado nuevas especificaciones al modelo agregando más funcionalidades para el procesamiento de archivos con propiedades que necesitan de otras funciones.

Origen del Modelo DOM

Los elementos que componen al archivo XML son tratados por el modelo DOM como objetos. Los atributos que contienen estos objetos son tratados por el model DOM como cadenas de caracteres o strings, por lo que tenemos que tomarlo en cuenta al realizar operaciones con los datos recién obtenidos del archivo de entrada.

Los atributos en el modelo DOM también pueden ser tratados como nodos en vez de strings aunque es bastante inusual hacerlo.

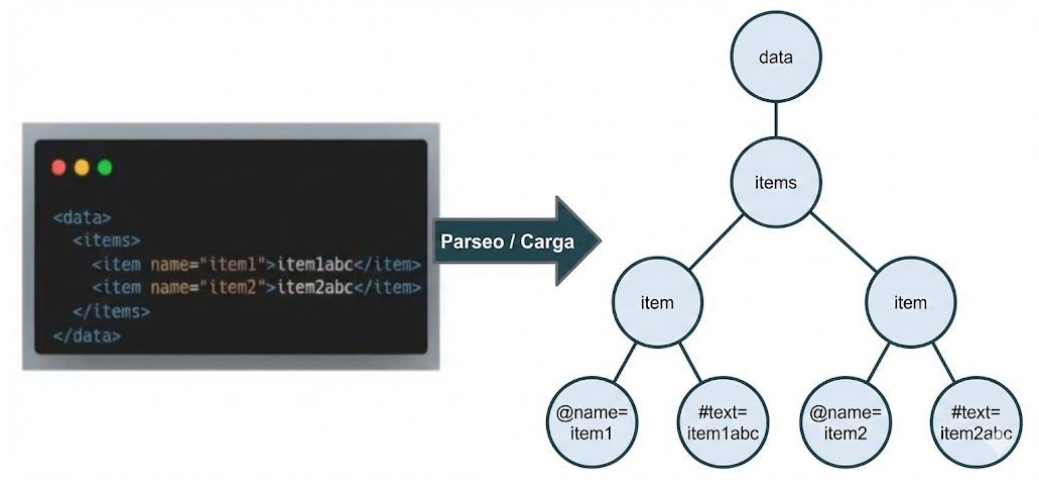
Objetos en el modelo DOM

Interfaz	Sección	Propósito
DOMImplementation	Objetos DOMImplementation	Interfaz para las implementaciones subyacentes.
Node	Objetos Nodo	Interfaz base para la mayoría de objetos en un documento.
NodeList	Objetos NodeList	Interfaz para una secuencia de nodos.
DocumentType	Objetos DocumentType	Información acerca de la declaraciones necesarias para procesar un documento.

Objetos en el modelo DOM

Document	Objetos Documento	Objeto que representa un documento entero.
Element	Objetos Elemento	Nodos elemento en la jerarquía del documento.
Attr	Objetos Atributo	Nodos de los valores de los atributos en los elementos nodo.
Comment	Objetos Comentario	Representación de los comentarios en el documento fuente.
Text	Objetos Texto y CDATASection	Nodos con contenido textual del documento.
ProcessingInstruction	Objetos ProcessingInstruction	Representación de instrucción del procesamiento.

Ejemplo de archivo de XML de Entrada

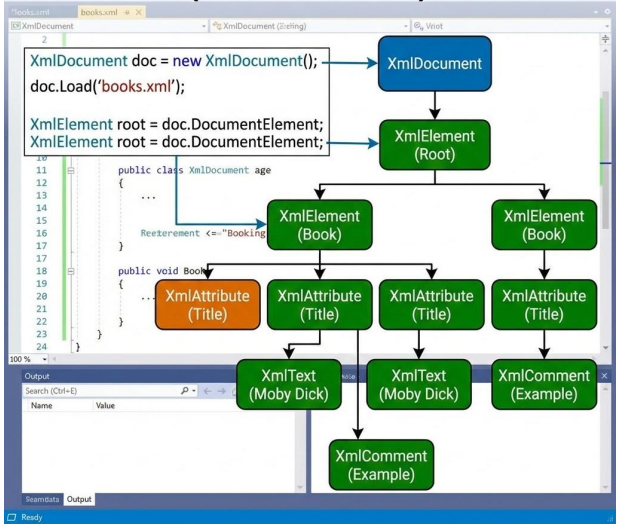


C# convierte el texto del archivo XML en una **estructura de árbol** en memoria (conocida como **DOM**). Esto nos permite navegar por los datos jerárquicamente, accediendo a 'padres' e 'hijos'.

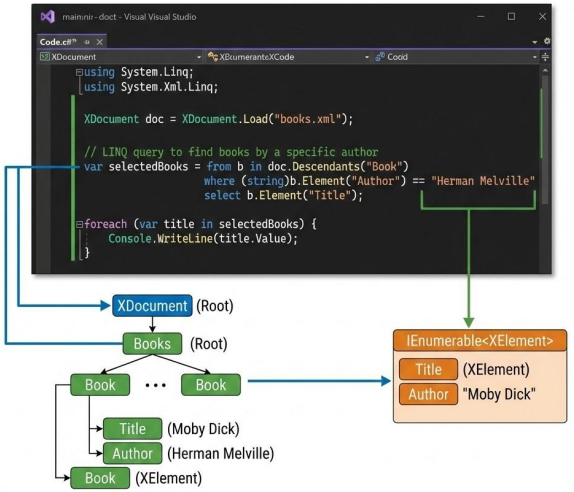
Al cargar el archivo, las librerías parsean el XML y construyen el **Árbol de Nodos**. Esta representación en memoria (DOM) permite manipular cada etiqueta y atributo como un objeto.

Procesamiento de Datos XML

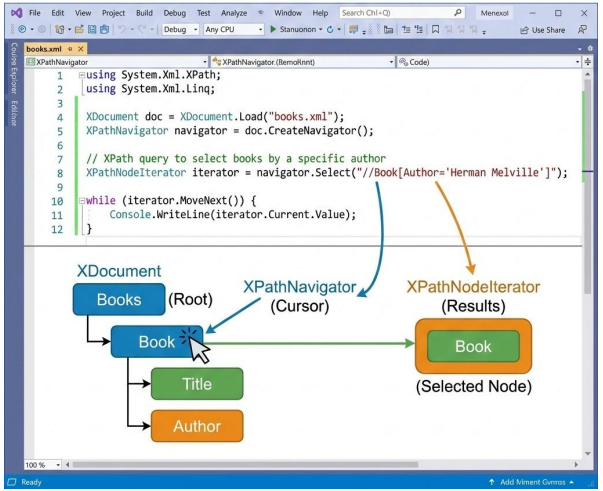
XmlDocument Class in C# (DOM Structure)



XDocument Class with LINQ in C# (LINQ to XML)



XPathNavigator Class in C# (XPath Navigation)



En C# Podemos usar estas librerías.

XmlDocument

Versión Legacy

```
// Para usar XmlDocument, necesitamos el namespace System.Xml
// Opcion clascia Legacy con W3C DOM (System.Xml)
using System.Xml;
```

XmlDocument es parte de **System.Xml**, que es la api tradicional para trabajar con **XML en C#**. Es más verbosa y menos intuitiva que XDocument, pero sigue siendo ampliamente utilizada, especialmente en proyectos más antiguos.

```
public void CrearXMLConXmlDocument()
{
    XmlDocument doc = new XmlDocument();

    // Declaración
    XmlDeclaration xmlDeclaration = doc.CreateXmlDeclaration("1.0", "UTF-8", null);
    XmlElement root = doc.DocumentElement;
    doc.InsertBefore(xmlDeclaration, root);

    // Elemento Raíz
    XmlElement elementoRaiz = doc.CreateElement("Biblioteca");
    doc.AppendChild(elementoRaiz);

    // Crear un Libro
    XmlElement libro = doc.CreateElement("Libro");
    libro.SetAttribute("id", "1");

    XmlElement titulo = doc.CreateElement("Titulo");
    titulo.InnerText = "Cien Años de Soledad";
    libro.AppendChild(titulo);

    XmlElement autor = doc.CreateElement("Autor");
    autor.InnerText = "Gabriel García Márquez";
    libro.AppendChild(autor);

    // Agregar libro a la raíz
    elementoRaiz.AppendChild(libro);

    doc.Save("EjemplosXML/biblioteca_dom.xml");
}
```

XDocument

```
// Para usar XDocument, necesitamos el namespace System.Xml.Linq
using System.Xml.Linq;
```

XDocument es parte de **System.Xml.Linq**, que es una API moderna para trabajar con **XML en C#.** Es más fácil de usar y tiene soporte para **LINQ**, lo que facilita la consulta y manipulación de XML.

```
1  public void CrearXMLConXDocument()
2  {
3      // Construcción funcional (Functional Construction)
4      XDocument doc = new XDocument(
5          new XDeclaration("1.0", "utf-8", "yes"), // opcional, pero recomendado para especificar la versión y codificación
6          new XComment("This is a comment"),
7          new XElement("Biblioteca",
8              new XElement("Libro", new XAttribute("id", "1"),
9                  new XElement("Titulo", "El Principito"),
10                 new XElement("Autor", "Antoine de Saint-Exupéry")
11             ),
12             new XElement("Libro", new XAttribute("id", "2"),
13                 new XElement("Titulo", "1984"),
14                 new XElement("Autor", "George Orwell")
15             )
16         )
17     );
18
19     doc.Save("EjemplosXML/XDocument.xml");
20     Console.WriteLine("XML creado con XDocument.");
21 }
```

XPathNavigator

```
// Para usar XPathNavigator, necesitamos el namespace System.Xml.XPath
using System.Xml.XPath;

```

XPathNavigator es parte de **System.Xml.XPath,** que se utiliza para **navegar y consultar XML** utilizando **expresiones XPath**. Es útil para consultas complejas, pero no es tan fácil de usar como XDocument para la creación y manipulación de XML. Con XPathNavigator por lo general no se crea y manipula XML, es más que todo para realizar consultas que pueden llegar a ser complejas.

```dotnet
public void Ejecutar()
{
    if (!File.Exists(rutaArchivo))
    {
        Console.WriteLine("¡Error! No encontré el archivo tienda_compleja.xml");
        return;
    }

    // 1. Cargar el documento en modo solo lectura (Optimizado)
    XPathDocument doc = new XPathDocument(rutaArchivo);
    XPathNavigator nav = doc.CreateNavigator();

    Console.WriteLine("--- ANÁLISIS DE TIENDA CON XPATH ---");

    // --- CASO 1: Filtrado Lógico Complejo ---
    // "Dame los productos que sean 'Procesador' Y que estén en 'oferta'"
    Console.WriteLine("\n1. Procesadores en Oferta:");

    XPathNodeIterator iterador = nav.Select("//Producto[@categoria='Procesador' and @oferta='true']/Nombre");

    while (iterador.MoveNext())
    {
        Console.WriteLine($"   -> {iterador.Current?.Value}");
    }

    // --- CASO 2: Matemáticas en el XML (Valor del Inventario) ---
    // ¿Cuánto dinero tengo en total en GPUs? (Precio * Stock de cada una)
    // Nota: XPath 1.0 no soporta multiplicacion de arrays directos, asi que iteramos
    Console.WriteLine("\n2. Valor total del inventario de Gráficas (Precio * Stock):");

    XPathNodeIterator graficas = nav.Select("//Producto[@categoria='Grafica']");
    double totalGraficas = 0;

    while (graficas.MoveNext())
    {
        // Movemos el cursor relativo al nodo actual
        double precio = double.Parse(graficas.Current.SelectSingleNode("Precio")?.Value ?? "0");
        int stock = int.Parse(graficas.Current.SelectSingleNode("Stock")?.Value ?? "0");
        totalGraficas += (precio * stock);
    }
    Console.WriteLine($"   -> Total invertido en GPUs: ${totalGraficas}");

    // --- CASO 3: Búsqueda por Valor Numérico (Range) ---
    // "Dame productos con precio mayor a 500 USD"
    Console.WriteLine("\n3. Productos de Gama Alta (> $500):");

    XPathNodeIterator caros = nav.Select("//Producto[Precio > 500]");

    while (caros.MoveNext())
    {
        string nombre = caros.Current.SelectSingleNode("Nombre")?.Value ?? "";
        string precio = caros.Current.SelectSingleNode("Precio")?.Value ?? "";
        Console.WriteLine($"   -> {nombre} ({precio})");
    }

    // --- CASO 4: Función de Agregación (Count) ---
    // Contar cuántos productos tienen stock critico (menos de 5 unidades)
    Console.WriteLine("\n4. Alertas de Stock Bajo (< 5 unidades):");

    // La función count() devuelve un número directamente
    double cantidadCritica = (double)nav.Evaluate("count(//Producto[Stock < 5])");

    Console.WriteLine($"   -> Hay {cantidadCritica} productos con stock critico.");
}
```

Conceptos Clave Aprendidos

- ★ **Conceptos y fundamentos de XML**
  Comprensión de qué es XML y su función en el intercambio y almacenamiento de datos. Reconocimiento de su estructura jerárquica basada en etiquetas, atributos y nodos, así como la importancia de seguir estándares para garantizar la interoperabilidad.
- ★ **Procesamiento con el modelo DOM**
  Aprendizaje del uso del modelo **Document Object Model (DOM)** para cargar, recorrer y modificar un documento XML completo en memoria, accediendo a sus elementos y atributos como objetos estructurados.
- ★ **Lectura y escritura de XML**
  Implementación de técnicas para **leer** archivos XML desde diversas fuentes y **escribir** nuevos documentos manteniendo la estructura correcta y la coherencia de los datos.

Valores y Actitudes de Esta Semana

- ★ **Rigor analítico:** Verificar que la estructura y contenido del XML sean correctos y cumplan con los estándares establecidos.
- ★ **Precisión:** Mantener la integridad de los datos al leer, manipular o escribir documentos XML.
- ★ **Responsabilidad:** Garantizar que el procesamiento de la información sea seguro, evitando pérdidas o corrupción de datos.
- ★ **Orden y organización:** Respetar las jerarquías y convenciones de nomenclatura para facilitar el mantenimiento del código y de los archivos XML.
- ★ **Adaptabilidad:** Elegir la técnica de procesamiento más adecuada (DOM, ElementTree, XPath) según el tamaño del archivo y los requisitos del proyecto.

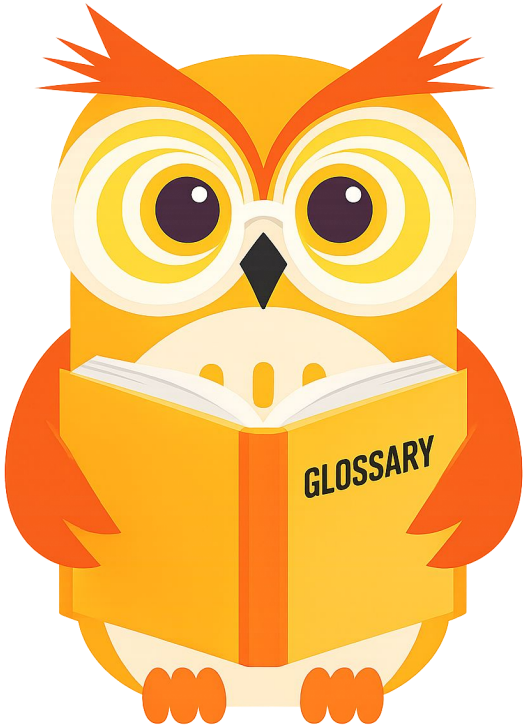
Referencias

- DesarrolloWeb.com. (s.f.). *Manual de XML*. Recuperado el 24 de mayo de 2025, de https://desarrolloweb.com/manuales/manual-xml.html

- AprendeAProgramar.com. (s.f.). *Introducción a XML*. Recuperado el 24 de mayo de 2025, de https://www.aprendeaprogramar.com/index.php?option=com_content&view=article&id=483

- MDN Web Docs. (s.f.). *XML*. Mozilla Foundation. Recuperado el 24 de mayo de 2025, de https://developer.mozilla.org/es/docs/Web/XML

Ejemplo Práctico



Practiquemos lo Aprendido

¿Dudas?

!Gracias por su Atención¡

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA QUE TE SURJA EN LA SEMANA

ESCUELA DE INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

ECYS | DTT DESARROLLO DE TRANSFERENCIA TECNOLÓGICA

Nombre de la actividad:	Actividad Semana
Cantidad de participantes:	
Doy fe que esta actividad está planificada en dtt (Sí/No):	

Hora de inicio:	
Hora de fin:	
Duración (min):	

Participantes: llenar las siguientes cajas de texto (tomar información del chat del meet)